Appendix C. Algorithmic details

Algorithm 1 obtains the spare part of a PWP baseline after the standard panels are placed, Algorithm 2 parameterizes the spare part into the gene pool of the baseline, and Algorithm 3 combines the sequence of PWP segments encoded in a chromosome into standard panels using the combination box (Sections 4.2.2 and 4.2.3). In Algorithm 3, chrom denotes the input chromosome (the sorted IDs of segments), iniListW_segs the list of segment widths before sorting, and w_StdPnl the width of the standard panel; tmp_box is a temporary box used to try fitting PWP segments, boxes records the boxes that successfully store segments, and tmp_ids and boxes_ids track the corresponding segment IDs.

Algorithm 1: Obtaining of the spare part of the PWP baseline

Input : Len_{PWP}, $w_{seg}^{\text{code,min}}$, w_{StdPnl}
Output : $nStdPnls$, L_{spare}

if $Len_{PWP} \le (w_{seg}^{\text{code,min}} + w_{StdPnl})$ **then**
  $nStdPnls \leftarrow 0$
  $L_{spare} \leftarrow Len_{PWP}$
else
  $nStdPnls \leftarrow \lfloor Len_{PWP} / w_{StdPnl} \rfloor$
  $L_{spare} \leftarrow Len_{PWP} \bmod w_{StdPnl}$
  if $L_{spare} \le w_{seg}^{\text{code,min}}$ **then**
    $nStdPnls \leftarrow nStdPnls - 1$
    $L_{spare} \leftarrow L_{spare} + w_{StdPnl}$
  end if
end if
return ($nStdPnls$, L_{spare})

Algorithm 2: The parameterization of the spare part of the PWP baseline

Input : L_{spare}, $w_{seg}^{\text{code,min}}$, w_{StdPnl}, $w_{seg,set}^{step}$

Output : *listSegs*

if $L_{spare} \leq 2 \times w_{seg}^{\text{code,min}}$ **then**

    listSegs $\leftarrow$ list[list[L_{spare}]]

else

    listSegs = list[]

    for w in range($w_{seg}^{\text{code,min}}, w_{seg,set}^{step}, w_{seg,set}^{step}$) **do**

        $w_{spare,seg}^{1} = w$

        $w_{spare,seg}^{2} = L_{spare} - w$

        listSegs.append([$w_{spare,seg}^{1}, w_{spare,seg}^{2}$])

    end for

end if

return *listSegs*

Algorithm 3: The combination of PWP segments into standard PWP panels

Input : $chrom$, $iniListW_{segs}$, w_{stdPnl}
Output : $boxes_ids$
#*step* 1
Initialize $list_segs_wds \leftarrow$ list[]
for *id* **in** chrom **do**
    $list_segs_wds.append(iniListW_{segs}[id])$
end for
#*step* 2
Initialize $boxes \leftarrow$ list[]
Initialize $boxes_ids \leftarrow$ list[]
Initialize $tmp_box \leftarrow$ list[]
Initialize $tmp_ids \leftarrow$ list[]
for (*ct*, *w*) **in** enumerate(*list_segs_wds*) **do**
    $tmp_box.append(w)$
    $tmp_ids.append(chrom[ct])$
    if sum(tmp_box) $\le w_{stdPnl}$ **then**
        if *ct* = len(*list_segs_wds*)-1 **then**
            boxes.append(*tmp_box*)
            boxes_ids.append(*tmp_ids*)
        end if
    else
        $tmp_box.pop()$
        $tmp_ids.pop()$
        boxes.append(*tmp_box*)
        boxes_ids.append(*tmp_ids*)
        $tmp_box \leftarrow$ list[*w*]
        $tmp_ids \leftarrow$ list[*chrom*[*ct*]]
        if *ct* = len(*list_segs_wds*)-1 **then**
            boxes.append(*tmp_box*)
            boxes_ids.append(*tmp_ids*)
        end if
    end if
end for
return *boxes_ids*